CREDIT CARD APPROVAL PREDICTION SYSTEM

Machine Learning-Based Underwriting Automation

EDA Insights, Classifiers Evaluation, and Web Application Guide

Developer Documentation

Powered by Scikit-Learn, Pandas, NumPy, and Flask

June 2026

1. Project Overview

Banks and financial institutions process thousands of credit card applications daily. Manually auditing every profile is highly resource-intensive and prone to human error. This project implements an automated, machine-learning-driven underwriting system. By training predictive classification models on historical customer demographic and financial profiles, the system evaluates applicant parameters to instantly flag low-risk profiles for approval and identify high-risk accounts to mitigate potential defaults.

2. Workspace Architecture

```
creditcard_approval_project/
|-- static/
|   |-- css/style.css                # Premium dark-theme stylesheet
|   |-- images/
|       |-- approval_distribution.png # Distribution count chart
|       |-- income_distribution.png   # Income distribution plot
|       |-- education_vs_approval.png # Education vs approval plot
|       |-- income_type_vs_approval.png# Income source vs approval plot
|-- templates/
|   |-- home.html                   # HTML: Dashboard and User Form
|   |-- result.html                 # HTML: Decision display view
|-- app.py                          # Flask routing and model server
|-- model.py                        # Model training and export script
|-- eda.py                          # EDA generator script
|-- credit_model.pkl                # Production Random Forest Model
|-- application_record.csv          # Dataset: Applicant demographic details
|-- credit_record.csv               # Dataset: Monthly payment history
```

3. Feature Engineering & Mappings

The model is trained strictly on the 12 features present in the Flask application input form:

- Gender (CODE_GENDER): Encoded as Male (1) or Female (0).
- Own Car (FLAG_OWN_CAR) / Own Realty (FLAG_OWN_REALTY): Encoded as Yes (1) or No (0).
- Income Source (NAME_INCOME_TYPE): Working, Associate, Pensioner, State servant, Student.
- Education Level (NAME_EDUCATION_TYPE): Higher, Secondary, Incomplete, Lower, Academic.
- Marital Status (NAME_FAMILY_STATUS) / Housing Type (NAME_HOUSING_TYPE).
- Occupation Sector (OCCUPATION_TYPE): Mapped across 19 unique industrial segments.
- Annual Income (AMT_INCOME_TOTAL): Total annual earnings of the client.
- Age (DAYS_BIRTH): Computed from years to negative days relative to evaluation.
- Employment Duration (DAYS_EMPLOYED): Computed to negative days; 365243 for unemployed.
- Family Members (CNT_FAM_MEMBERS): Total count of members in the household.

4. Classification Algorithms Performance

The classifiers were evaluated using a stratified 80/20 train-test split on 36,457 integrated accounts. Minor payment delays (under 30 days) are grouped as Approved, and overdue durations exceeding 30 days are classified as High-Risk

(Rejected).

Model	Accuracy	Precision (C1)	Recall (C1)	F1-Score (C1)
Logistic Regression	88.23%	0.00%	0.00%	0.00%
Decision Tree	88.30%	50.46%	32.17%	39.29%
Random Forest (Production)	**88.95%**	**54.87%**	**34.15%**	**42.10%**
Gradient Boosting (Fallback)	88.25%	60.00%	0.35%	0.70%

5. System Operation & Execution Guide

Follow these steps to operate or modify the system:

• **STEP 1: Activate Virtual Environment**

Open terminal inside the project folder and activate the environment:

```
PowerShell:  .\venv\Scripts\Activate.ps1
CMD:         .\venv\Scripts\activate.bat
```

• **STEP 2: Re-generate Visual Insights (EDA)**

Re-process customer files and update saved count and distribution charts:

```
Command:     python eda.py
```

• **STEP 3: Retrain Classifiers**

Re-train all four algorithms and export the updated best model:

```
Command:     python model.py
```

• **STEP 4: Start Underwriting Server**

Run the Flask server and open the web dashboard:

```
Command:     python app.py
```

Open your web browser and navigate to http://127.0.0.1:5000/